

Elevator Control System — Complete Implementation Plan & Documentation

Detailed end-to-end plan, code samples, file templates, tests, CI, deployment, and Cursor AI prompts.

Table of Contents

1	1. Project Summary & Goals
2	2. Tech Stack, Tools & Project Setup
3	3. Architecture & Folder Structure
4	4. Internationalization (i18n) Adapter — Detailed
5	5. Theming & Styling — Detailed
6	6. UI Components — Detailed file-by-file
7	7. Elevator Logic — Single and Multi Elevator (detailed algorithms)
8	8. State Management & Stores (Zustand)
9	9. Scheduler Implementation & Complexity
10	10. Animations, Sound & Extras
11	11. Tests — Unit & Integration (code samples)
12	12. CI / GitHub Actions (YAML)
13	13. Scripts, package.json, and tooling config
14	14. Deployment (Vercel/Netlify)
15	15. Issue list / Sprint board / Cursor AI prompts
16	16. Acceptance Criteria, Checklists, and Estimates
17	17. How to add translations & switch adapters (step-by-step)
18	Appendix: Full-file templates and code snippets

1. Project Summary & Goals

Build a responsive, modular, and testable web application that simulates one or more elevators in one or more buildings. The application must support internationalization (with an engine-agnostic adapter), theme-based styling, a clean state-management solution, queue-based elevator scheduling, multi-elevator assignment (choose the best elevator), animations, sound cues, tests, CI and a deployable demo.

2. Tech Stack, Tools & Project Setup

Recommended stack and tools: - Frontend: React + TypeScript, Vite (template: react-ts) - Styling: styled-components + theme-ui (or styled-system primitives) - State: Zustand (simple and testable) or React Context (clear API) - i18n: Adapter pattern (react-i18next implementation provided) - Animation: framer-motion (optional) or CSS transitions - Tests: Jest + @testing-library/react + ts-jest - CI: GitHub Actions - Optional: Storybook for component catalog - Optional: html2canvas for downloading images, and small audio asset for arrival sound

Project Setup (commands)

```
# Create project
npm create vite@latest elevator-app -- --template react-ts
cd elevator-app
npm install

# Install dependencies (example)
npm install react-i18next i18next styled-components @types/styled-components theme-ui zustand framer-motion html2canvas
npm install -D jest @testing-library/react @testing-library/jest-dom ts-jest @types/jest babel-jest eslint prettier

# Scripts to add to package.json
- dev: vite
- build: vite build
- preview: vite preview
- test: jest
- lint: eslint . --ext .ts,.tsx
```

3. Architecture & Folder Structure

Logical folder structure (explain purpose of each):

```
src/
■■■ components/      # Reusable UI components (Elevator, Floor, Building, Header, Buttons)
■■■ pages/           # Page-level entry points (App, DemoPage)
■■■ hooks/           # Custom hooks (useElevatorLogic, useAnimation)
■■■ stores/          # Zustand stores and scheduler functions
■■■ i18n/            # Translator interface + adapter implementations
■■■ locales/         # Translation JSON files (en, sq, etc.)
■■■ theme/           # Theme tokens and ThemeProvider wrapper
■■■ assets/          # Images, audio files (arrival sound)
■■■ tests/           # Unit & integration tests
■■■ utils/           # Utility helpers (timing, math)
■■■ main.tsx         # App bootstrap
```

Data models (TypeScript types)

```
export type ElevatorId = string;
export type FloorNumber = number;

export type ElevatorStatus = 'idle' | 'moving' | 'stopped' | 'doorOpen';

export interface Elevator {
  id: ElevatorId;
  currentFloor: FloorNumber;
  targetQueue: FloorNumber[]; // FIFO queue of stops
  status: ElevatorStatus;
  direction: 'up' | 'down' | 'idle';
  speedMsPerFloor?: number; // configurable
}

export interface BuildingConfig {
  id: string;
  floors: number;
  elevators: Elevator[];
}
```

4. Internationalization (i18n) Adapter — Detailed

Goal: Provide a stable translator interface that other components import from, while the actual engine (react-i18next, lingui, etc.) is implemented behind an adapter. This allows future swapping with minimal changes. Interface (src/i18n/translator.ts): - init(options?: {lng: string}): Promise - t(key: string, opts?: any): string - useT(): (key: string, opts?:any) => string // react hook - changeLanguage(lng: string): Promise - getAvailableLanguages(): string[]

```
// src/i18n/translator.ts
export type Translator = {
  init: (options?: {lng?: string}) => Promise<void>;
  t: (key: string, opts?: any) => string;
  useT: () => (k: string) => string;
  changeLanguage: (lng: string) => Promise<void>;
  getAvailableLanguages: () => string[];
};
```

```
// export a default adapter instance (initially pointing to react-i18next adapter)
import * as ReactI18nAdapter from './reactAdapter';
const adapter: Translator = ReactI18nAdapter.createAdapter();
export default adapter;
```

How to add new language (for non-technical users)

1. Create a new folder under src/locales, e.g., src/locales/de/translation.json
2. Add key-value pairs in JSON (copy from en and translate values).
3. Commit the JSON file and push to repo – the app will pick it up (if using runtime import) or will be available at build time.
4. To expose in UI, add the language code to getAvailableLanguages() in adapter configuration.

5. Theming & Styling — Detailed

Use Styled Components with a theme object. Optionally provide theme-ui primitives for design tokens and responsive constraints. Files: - src/theme/index.ts (theme tokens) - src/theme/ThemeProvider.tsx (wraps styled-components ThemeProvider) Design tokens include colors, spacing, typography, breakpoints, sizes, and motion timings. Provide small library of primitives: Box, Flex, Button that map to styled-system props (or manually typed).

```
// src/theme/index.ts
export const theme = {
  colors: {
    primary: '#028AFB',
    secondary: '#0D74C9',
    background: '#0f172a',
    surface: '#0b1220',
    text: '#e6eef8',
  },
  space: [0, 4, 8, 12, 16, 24, 32, 48],
  fonts: { body: 'Manrope, system-ui, sans-serif' },
  breakpoints: ['480px', '768px', '1024px'],
  elevator: { speedMsPerFloor: 500, doorOpenMs: 800 }
};
```

6. UI Components — Detailed file-by-file

Each component must be small, focused, and testable. Provide prop types and clear responsibilities. Key components: - Header (language switcher) - Building (renders floors and elevators) - Floor (display floor number and call buttons) - ElevatorCar (visual elevator, position based on currentFloor) - ElevatorShaft (container for ElevatorCar movements) - FloorButton (call up/down or call single)

Component: Building.tsx (contract)

Props:

- floors: number
- elevators: { id: string; initialFloor: number }[]

Responsibilities:

- Render list of Floor components from top to bottom
- Render ElevatorShaft with ElevatorCar for each elevator
- Connect to store to receive elevator positions and status
- Dispatch call events when floor button clicked

Component: ElevatorCar.tsx (behavior)

```
// ElevatorCar: receives elevator object {id,currentFloor,status} and renders at appropriate Y offset
// Calculate Y by (floorIndex / (floors - 1)) * shaftHeight and apply CSS transform translateY
// Add aria-labels for accessibility and a label showing current floor
```

7. Elevator Logic — Single and Multi Elevator (detailed algorithms)

This section explains the queue processing, movement simulation and multi-elevator scheduling in detail.

State per elevator:

- currentFloor (number)
- targetQueue (number[]) // FIFO
- status: 'idle' | 'moving' | 'doorOpen' | 'stopped'
- direction: 'up' | 'down' | 'idle'

Enqueue behavior:

- On call(floor): push floor to targetQueue (if not present) -> duplicate requests can be ignored or merged

Processing loop (processNext):

1. If targetQueue.length === 0, set status = 'idle', return
2. const nextFloor = targetQueue.shift()
3. Set status = 'moving', set direction = (nextFloor > currentFloor ? 'up' : 'down')
4. Simulate travel: const floorsToMove = Math.abs(nextFloor - currentFloor)
 - Use an async helper that resolves after floorsToMove * speedMsPerFloor
 - For better animation, update currentFloor as fraction using requestAnimationFrame or setInterval to animate
5. On arrival: set currentFloor = nextFloor; set status='doorOpen'; await doorOpenMs; set status='stopped'; then

Important: Use clear cancellation and guards when new commands arrive while moving (e.g., adding stops in same

Goal: Given a requestFloor, choose elevator with minimum estimated arrival time.

Estimate function estimateTime(elevator, requestFloor):

- If elevator.targetQueue.length === 0:
 - time = abs(elevator.currentFloor - requestFloor) * speedMsPerFloor
- Else:
 - simulate walking through elevator.targetQueue starting from elevator.currentFloor:
 - pos = elevator.currentFloor
 - time = 0
 - for each q in elevator.targetQueue:
 - time += abs(pos - q) * speedMsPerFloor + doorOpenMs
 - pos = q
 - time += abs(pos - requestFloor) * speedMsPerFloor

Return time

Scheduler:

- Compute estimateTime for each elevator
- Choose elevator with min time
- Breaking ties: prefer elevator with same direction and that will pass requestFloor, otherwise with smallest q
- Enqueue request to chosen elevator's targetQueue

Complexity: $O(E * Q)$ where E = number of elevators and Q = average queue length. For typical small E and Q , this

8. State Management & Stores (Zustand)

Use a single store per building or a central store keyed by buildingId. Expose pure functions to mutate state for easier testing. Store API (suggested): - getState(): current raw state (for tests) - subscribe() - enqueueRequest(buildingId, elevatorId, floor) - callElevator(buildingId, floor) // scheduler finds elevator and enqueues - processNext(elevatorId) - setElevatorConfig(elevatorId, key, value) // for runtime config
Example store file: src/stores/elevatorStore.ts Ship small pure utility functions separately: src/stores/scheduler.ts with pure estimateTime and pickBestElevator functions to unit test easily.

9. Scheduler Implementation & Complexity

Implementation details and edge cases to code defensively for: - Duplicate requests: if same floor already in queue, skip adding (or update timestamp) - Request cancellation (optional): allow users to cancel requests until elevator stops - Elevator breakdown / maintenance mode: mark elevator unavailable and reassign queued requests - Priorities: VIP requests or emergency mode (override queue ordering)

Performance: - For small numbers ($E \leq 10$, $Q \leq 20$) simple $O(E*Q)$ algorithm is fine - For larger scale, consider priority queue, spatial indexing, or more advanced heuristics

10. Animations, Sound & Extras

Animations: - Prefer framer-motion for smooth spring-like movement or use CSS transition for transform
translateY - ElevatorCar position computed from currentFloor and total floors transform:
 $\text{translateY}(\text{calc}((\text{currentFloor} / (\text{floors}-1)) * \text{shaftHeight}))$ Sound: - Provide short 'ding' audio when elevator arrives. Use HTMLAudioElement to play on arrival (ensure user gesture allowed on mobile). - Keep asset small (e.g., 5-10 KB). Place in src/assets/sounds/arrival.mp3 Download snapshot: - Use html2canvas to render building element and provide download link as PNG. Accessibility: - Aria labels for floor buttons and elevator status - Keyboard navigation: focusable floor buttons and Enter to call

11. Tests — Unit & Integration (code samples)

Use Jest + React Testing Library with fake timers to simulate time-based movement deterministically. Key tests to include: - enqueueRequest appends floor correctly and avoids duplicates - processNext moves elevator through queue and updates currentFloor - scheduler.pickBestElevator chooses correct elevator in multiple scenarios - Component test: clicking floor button dispatches call Example unit test for scheduler.pickBestElevator (pseudocode provided)

```
// tests/scheduler.test.ts
import { estimateTimeToServe, pickBestElevator } from '../src/stores/scheduler';

test('picks idle closest elevator', () => {
  const elevators = [
    { id: 'e1', currentFloor: 1, targetQueue: [] },
    { id: 'e2', currentFloor: 6, targetQueue: [] }
  ];
  const chosen = pickBestElevator(elevators, 4, 500, 800);
  expect(chosen.id).toBe('e2'); // closer to floor 4 (distance 2 vs 3)
});
```

12. CI / GitHub Actions (YAML)

name: CI

on:

push:

branches: [main, dev]

pull_request:

branches: [main]

jobs:

build-and-test:

runs-on: ubuntu-latest

steps:

- uses: actions/checkout@v4
- name: Use Node.js
 - uses: actions/setup-node@v4
 - with:
 - node-version: 20
- name: Install dependencies
 - run: npm ci
- name: Run lint
 - run: npm run lint
- name: Run tests
 - run: npm test -- --coverage

13. Scripts, package.json, and tooling config

Suggested package.json scripts:

```
"scripts": {  
  "dev": "vite",  
  "build": "vite build",  
  "preview": "vite preview",  
  "test": "jest --config jest.config.js --runInBand",  
  "test:watch": "jest --watch",  
  "lint": "eslint . --ext .ts,.tsx",  
  "format": "prettier --write ."  
}  
jest.config.js: configure ts-jest, testEnvironment: jsdom  
.eslintrc.js & .prettierrc: standard config for TypeScript + React
```

14. Deployment (Vercel/Netlify)

Vercel quick deploy: - Connect GitHub repo, set build command ``npm run build`` and output directory ``dist``.
- Ensure environment variables (if any) set in project settings. Netlify quick deploy: - Connect repo, set build command ``npm run build``, publish directory ``dist``.

15. Issue list / Sprint board / Cursor AI prompts

- 1 **ISSUE-0: Project scaffold:** Create Vite TypeScript project, add ESLint, Prettier, Jest. Commit initial scaffold.
- 2 **ISSUE-1: i18n adapter:** Implement translator interface & react-i18next adapter. Add en & sq locales.
- 3 **ISSUE-2: Theme provider:** Create theme tokens and ThemeProvider, wrap app.
- 4 **ISSUE-3: Building UI:** Implement Building, Floor, ElevatorCar components and layout.
- 5 **ISSUE-4: ElevatorStore (single):** Implement Zustand store for single elevator queue and movement.
- 6 **ISSUE-5: Scheduler (multi):** Implement pickBestElevator, estimateTime and integration with store.
- 7 **ISSUE-6: Tests:** Add unit tests for scheduler and elevator store, add component tests.
- 8 **ISSUE-7: CI:** Add GitHub Actions CI YAML to run lint & tests.
- 9 **ISSUE-8: Deploy:** Deploy demo to Vercel and add demo link in README.

Cursor AI prompt template (use per issue):

Task: <title>
Goal: <goal statement>
Files to create/modify: <file list>
Details: <detailed instructions and code examples>
Acceptance criteria: <list of checks>
Commit message: <short message>

16. Acceptance Criteria, Checklists, and Estimates

Acceptance checklist (must pass before merging):

- [] Project starts with `npm run dev`
- [] i18n strings loaded and switchable
- [] Building UI renders with floors and elevator(s)
- [] Elevator moves visually and processes queue
- [] Scheduler selects nearest elevator in multi-elevator scenarios
- [] Tests added and passing (coverage goal: >70%)
- [] Linting and formatting applied
- [] Demo deployed and README updated

Estimated time (rough):

- Setup & i18n: 6-10 hours
- Theme & UI: 8-12 hours
- Single elevator logic: 6-10 hours
- Multi-elevator scheduler & tests: 10-16 hours
- Polish & deploy: 4-8 hours

Total: 34-56 hours

17. How to add translations & switch adapters (step-by-step)

Add translation (non-technical):

1. Create new folder `src/locales/<lang>/translation.json`
2. Copy keys from `en translation.json` and translate values
3. Commit & push

Switch adapter (developer):

1. Implement new adapter in `src/i18n/<yourAdapter>.ts` implementing `Translator` interface
2. Update `src/i18n/translator.ts` to import and export the new adapter (or make adapter selection by env var)
3. Ensure tests still use the adapter mock or test wrapper

Appendix: Full-file templates and code snippets

This appendix contains ready-to-copy templates for key files. Paste into project and adapt names/paths.

```
// src/stores/elevatorStore.ts (template - simplified)
import create from 'zustand';

export type Elevator = {
  id: string;
  currentFloor: number;
  targetQueue: number[];
  status: 'idle'|'moving'|'doorOpen'|'stopped';
  direction: 'up'|'down'|'idle';
  speedMsPerFloor?: number;
};

type State = {
  elevators: Record<string, Elevator>;
  enqueue: (elevatorId: string, floor: number) => void;
  callElevator: (floor: number) => void;
  processNext: (elevatorId: string) => void;
};

export const useElevatorStore = create<State>((set, get) => ({
  elevators: {},
  enqueue: (elevatorId, floor) => {
    set(state => {
      const elev = state.elevators[elevatorId];
      if (!elev) return state;
      if (elev.targetQueue.includes(floor)) return state;
      return { elevators: { ...state.elevators, [elevatorId]: { ...elev, targetQueue: [...elev.targetQueue, floor] } } };
    });
  },
  callElevator: (floor) => {
    // scheduler.pickBestElevator should be used here
  },
  processNext: (elevatorId) => {
    // implement queue processing similar to docs
  }
}));
```

End of documentation